

Group ID: 02

RELABTeams

GROUP LEAD: J. SIVA SAMPREETH



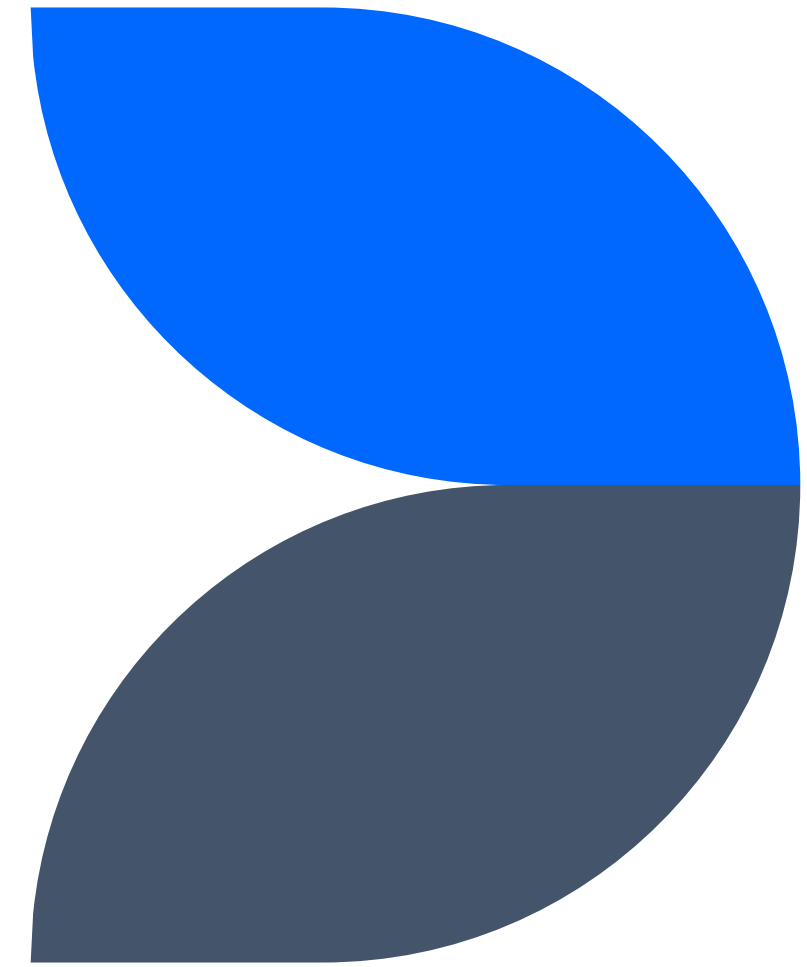
PROJECT BUSINESS IDEA

- Our platform lets anyone start or join real projects with people worldwide, ask doubts inside the project, and turn that work into a verified portfolio. You can create up to three projects for free; after that, take a paid plan for more projects and extra features. Recruiters can make a company profile, post jobs, search by skills, view real contributions, shortlist, message candidates, and schedule interviews; they pay per listing or via a subscription. Admins keep things clean and safe: verify recruiters and mentors, approve/flag job posts, review reports, remove spam, ban bad actors, and manage payments, subscriptions, and analytics. This helps students learn faster and companies hire better, while we track free-to-paid conversion, active projects, doubt response time, and job fill rate.



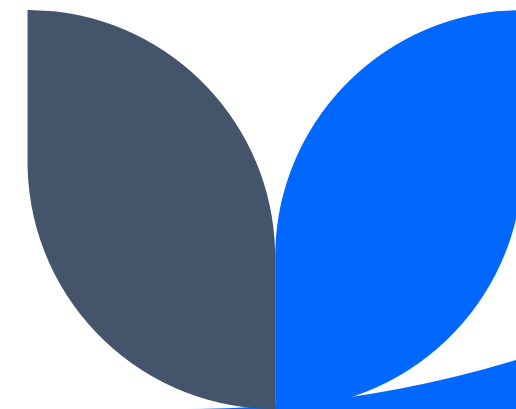
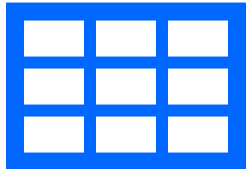
Team

- Siva Sampreeth Josyula
- Venkata Satya Surya Kaki
- Eswar babu Ettaligalla
- Srihesh Kothapalli
- Shiva Sai prasad Jadhav



THE RELABTeams

- Node.js + Express server with EJS server-side rendering, MongoDB/Mongoose for persistence, session-based authentication and role-based authorization enforced on sensitive endpoints (student/recruiter/admin).
- Server-side application flow and validation (POST /apply-job): session auth check, jobId ObjectId validation, required resume file, remove invalid uploads, duplicate-application prevention, persist JobApplication with relative uploads/... path, create Notification, upsert UserMetrics, and structured error handling and diagnostic logging.



RelabTeams Work Flow

- Project workflows : create and join projects; client-side join confirmation modal; hosted projects, capacity checks and leader/member roles.
- Job application flow: client sends FormData with jobId and resume; server POST /apply-job validates session, jobId, file presence and type, prevents duplicates, persists JobApplication, creates Notification, and increments UserMetrics.
- Admin features: student management UI (admin-stud.ejs) with instant client-side search and column sorting, safe injection of studentsData, and activePage wiring for sidebar highlights.

Dom Validation Ajax And Async functionalities



Job creation validation

- Files: rec-jobs.ejs
- Client-side checks for title, description, pay, and skills; live character counters; Create button disabled until valid; AJAX POST to create job.

Job apply and resume upload

- Files: applyjobs.ejs, project_details.ejs, server route POST /apply-job in server.js
- Client builds FormData with jobId + resume, pre-send extension check (.pdf/.doc/.docx), async upload via fetch; on success flip card to Applied and disable inputs. Server enforces auth, ObjectId and file validation, deletes invalid uploads, prevents duplicates, persists application, creates notification and updates metrics.

Task review workflow

- Files: project_details.ejs
- Intercept Accept/Reject, collect feedback, send JSON { taskId, projectId, action, feedback } to /task/review-submission; update task status and feedback inline.



Join request approval flow

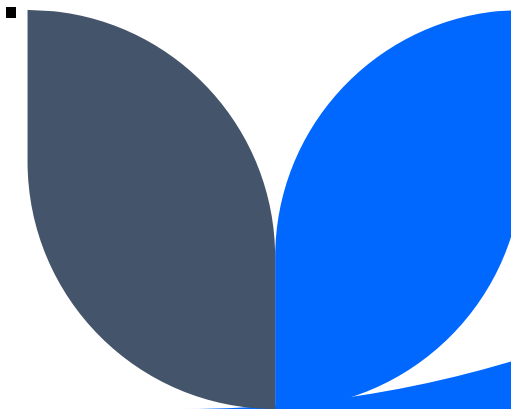
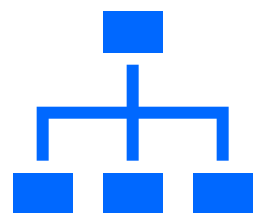
- Files: proj_notif.ejs
- Intercept Approve/Reject actions; POST to /approve-join-request or /reject-join-request; optimistic UI removal of request row and toast on success; non-blocking error handling for failures or already-processed responses.

Authentication form validation

- Files: login.ejs, signup.ejs
- Live per-field validation: email format, password policy, confirm-password matching, inline errors, submit disabled until fields pass, focus on first invalid on submit.

Project creation validation

- Files: create_proj.ejs (GET /e)
- Client checks: title ≥ 3 chars, capacity 1–20, topic required, deadline in future; single inline error display; Create disabled until valid.



Profile editing (chip lists)

- Files: profile.ejs
- Skills and Interests implemented as add/remove chips in the DOM; Save sends arrays to existing POST /profile without schema changes.

Admin students UX (search + sort)

- Files: admin-stud.ejs
- Instant client-side search and multi-column sort (Student, Hosted Projects, Tasks Completed) that composes with filters; guarded listeners so page works with empty studentsData; sort indicator present.

Profile editing (chip lists)

- Files: profile.ejs
- In profile.ejs we use fetch to send the updated profile payload (name, bio, skills and interests as arrays, and optional resume) to POST /profile.
- The client handles the JSON response to show a success or error toast and updates the UI instantly (no page reload).

Admin students UX (search + sort)

- Files: admin-stud.ejs
- Instant client-side search and multi-column sort (Student, Hosted Projects, Tasks Completed) that composes with filters; guarded listeners so page works with empty studentsData; sort indicator present.

Recruiter job management and notifications

- Files: rec-jobs.ejs and recruiter pages
- AJAX create/delete/toggle job actions, application and notification handling, optimistic updates and toasts; handlers bound safely after DOM injection.

Server-side authoritative checks & logging

- Files: server.js (notably POST /apply-job)
- Session auth, ObjectId validation, file presence , deletion of invalid uploads, duplicate-application prevention, persistence of JobApplication with relative path, creation of Notification, upsert UserMetrics, structured responses, diagnostic logging.

Project Summary

- Purpose: RELABTeams is a web platform for students to create and join projects, build skills, ask/answer doubts, and apply to jobs. Recruiters can post jobs and manage applicants. Admins oversee users and content.
- Project postings: students create projects with limits for capacity and deadlines; others can request to join. Leaders manage membership and tasks.
- Tasks and reviews: project owners can assign and review tasks; reviewers accept/reject with inline feedback.
- Jobs and applications: recruiters post jobs; students apply with resumes; recruiters manage applications and notifications.
- Admin functions: manage students, view metrics, and handle platform content.

